

Compiler Design Laboratory

Assignment V: Implementation of Recursive Descent Parser

Course: Compiler Design

Semester: VI B.Tech (CSE)

1. Objective

The objective of this experiment is to:

- Understand the working of **Recursive Descent Parsing**
- Implement a recursive descent parser for arithmetic expressions
- Detect syntax errors in input strings

2. Theory

Recursive descent parsing is a top-down parsing technique in which each non-terminal of the grammar is implemented as a recursive function. The parser starts from the start symbol and attempts to derive the input string using a leftmost derivation.

Important Notes:

- Recursive descent parsers cannot handle left-recursive grammars directly.
- The grammar must be rewritten to eliminate left recursion.

3. Given Grammar

Consider the following grammar for arithmetic expressions:

$$E \rightarrow E + T \mid T$$

$$T \rightarrow T * F \mid F$$

$$F \rightarrow (E) \mid id$$

Start Symbol: E

Terminals: $id, +, *, (,)$

Non-terminals: E, T, F

4. Grammar Transformation

The given grammar is left-recursive. After removing left recursion, the grammar becomes:

$$E \rightarrow TE'$$

$$E' \rightarrow +TE' \mid \epsilon$$

$$T \rightarrow FT'$$

$$T' \rightarrow *FT' \mid \epsilon$$

$$F \rightarrow (E) \mid id$$

This transformed grammar is suitable for recursive descent parsing.

5. Mapping Grammar to Functions

Non-terminal	Recursive Function
E	void E()
E'	void Ep()
T	void T()
T'	void Tp()
F	void F()

6. Parser Skeleton Code

```
#include <stdio.h>
#include <string.h>
#include <stdlib.h>

char input[100];
int pos = 0;

void E(); void Ep();
void T(); void Tp();
void F();

void match(char c) {
    if (input[pos] == c)
        pos++;
    else {
        printf("Syntax Error\n");
        exit(1);
    }
}
```

7. Lab Task

1. Implement all recursive functions based on the transformed grammar.
2. Use the `match()` function to match terminals.
3. Ensure that syntax errors are correctly reported.

8. Sample Test Cases

Valid Inputs:

```
id+id*id
(id+id)*id
id*id+id
```

Invalid Inputs:

```
id+*id
(id+id
id+
```

9. Driver Program

```
int main() {
    printf("Enter input string: ");
    scanf("%s", input);
    E();
    if (input[pos] == '\0')
        printf("String Accepted\n");
    else
        printf("String Rejected\n");
    return 0;
}
```

10. Expected Output

```
Enter input string: id+id*id
String Accepted
```

```
Enter input string: id+*id
Syntax Error
```

11. Submission Requirements

Students must submit:

- Complete C source code of the recursive descent parser
- Screenshots or text output for valid and invalid inputs

12. Learning Outcomes

After completing this experiment, students will be able to:

- Implement recursive descent parsers
- Remove left recursion from grammars
- Analyze syntax errors in expressions

— End of Lab Assignment —